Project Proposal

Project Name

JarvisOS (Working Title)

Vision

JarvisOS is a **self-hosted distributed AI operating system** that transforms one computer into an intelligent coordination server capable of managing AI models, devices, automation, memory, and user interactions.

Unlike a traditional AI assistant that only generates text, JarvisOS understands goals, creates execution plans, delegates tasks to connected devices, and verifies their completion.

The long-term objective is to create an extensible, local-first ecosystem where intelligence resides on a central server while execution is distributed across multiple devices.

Problem Statement

Current AI assistants suffer from several limitations:

- They cannot reliably control multiple personal devices.
- They are often cloud dependent.
- They lack persistent local intelligence.
- They have little awareness of user-owned hardware.
- They do not expose extensible architectures for developers.
- Most are designed as chatbots instead of operating systems.

JarvisOS aims to solve these problems.

Objectives

The project will:

- Operate completely on self-hosted hardware.
- Support multiple local AI models.
- Support multiple operating systems.
- Execute tasks across connected devices.
- Maintain persistent memory.

- Build long-term knowledge.
- Allow plugin-based expansion.
- Work over LAN and Internet.
- Remain model agnostic.

Core Philosophy

The project is **not** an AI assistant.

The project is an **AI Operating System**.

The LLM is considered one subsystem among many.

Everything is designed around modularity.

Design Principles

Local First

The system should function without cloud services.

Cloud providers may be added later as optional providers.

Model Agnostic

No component depends on a specific model.

Supported providers may include:

- Ollama
- llama.cpp
- Hugging Face Transformers
- Remote inference servers

Replacing one model should require no architectural changes.

Platform Independent

Supported operating systems:

- Linux
- Windows
- Android
- macOS (future)

Network Independent

Devices should communicate across:

- Local Area Network
- Internet
- VPN
- Secure tunnels

Plugin Based

All capabilities should be implemented as plugins.

Examples:

- Docker Plugin
- Browser Plugin
- Spotify Plugin
- Camera Plugin
- Filesystem Plugin

The core should not require modification when adding new features.

High-Level Architecture

JarvisOS consists of three layers.

Layer 1

Core Kernel

Responsible for:

- Configuration
- Logging
- Authentication
- Event Bus
- Task Queue
- Scheduler
- Planner
- Memory
- Knowledge
- Device Management
- Model Management
- Plugin Management

Layer 2

Providers

Examples:

- Ollama
- PostgreSQL
- Redis
- Vector Storage
- Speech Recognition
- Text-to-Speech

Providers implement interfaces defined by the core.

Layer 3

Agents

Agents execute work on devices.

Examples:

- Linux Agent
- Windows Agent
- Android Agent

Agents expose capabilities rather than operating-system-specific implementations.

System Components

Configuration Manager

Loads configuration from environment variables and configuration files.

Responsible for startup configuration.

Logger

Provides structured logging across all subsystems.

Event Bus

Acts as the communication backbone.

Subsystems communicate through events rather than direct dependencies.

Example events:

- DeviceConnected
- TaskCompleted
- ModelLoaded
- UserAuthenticated

Authentication Manager

Responsible for:

- User authentication
- Device authentication
- API tokens
- Session validation

Device Manager

Maintains connected devices.

Tracks:

- Device identity
- Status
- Health
- Capabilities
- Last heartbeat

Plugin Manager

Discovers and loads plugins.

Registers plugin capabilities.

Supports hot-loading in future versions.

Model Manager

Responsible for:

- Provider discovery
- Model discovery
- Loading
- Unloading
- Health monitoring
- Request routing

The Model Manager never performs inference itself.

Planner

Converts user goals into executable tasks.

Example:

User Goal

"Open Spotify on my phone."

Planner Output

Task 1

Target Device: Android

Capability: Open Application

Arguments: Spotify

Executor

Dispatches tasks to appropriate devices.

Tracks execution status.

Collects results.

Memory

Stores user-specific long-term information.

Examples:

- Preferences
- Devices
- Projects
- Habits
- Automation Rules

Memory is distinct from conversation history.

Knowledge

Stores external information.

Examples:

- PDFs
- Notes
- Images
- Videos
- Documentation
- Repositories

Knowledge forms the basis of Retrieval-Augmented Generation (RAG).

Device Agents

Every supported operating system runs a lightweight agent.

Responsibilities:

- Register with server
- Send heartbeats
- Advertise capabilities
- Execute authorized tasks
- Return execution results

Agents never make planning decisions.

Capabilities

Capabilities abstract operating-system functionality.

Examples:

Linux

- Shell
- Docker
- Filesystem
- Browser

Android

- Camera
- GPS
- Notifications
- Applications

Windows

- PowerShell
- Explorer
- Clipboard

The core interacts only with capabilities.

Tasks

Tasks are atomic units of execution.

Each task contains:

- Identifier
- Target Device
- Capability
- Parameters
- Status
- Result

Tasks remain deterministic.

AI does not execute tasks directly.

Artificial Intelligence

The AI subsystem is introduced only after infrastructure exists.

Responsibilities:

- Intent understanding
- Planning
- Tool selection
- Reasoning
- Summarization
- Conversation

AI never communicates directly with devices.

AI communicates only with the Planner and Model Manager.

Multi-Model Architecture

Different models perform different responsibilities.

Possible allocation:

Conversation

Qwen

Reasoning

Qwen

Vision

Qwen-VL

Speech Recognition

Whisper

Speech Synthesis

Piper

Embeddings

Nomic Embed

Future models should be replaceable without affecting the architecture.

Security

Every device authenticates before joining.

Communication is encrypted.

High-risk operations require confirmation.

Permissions determine accessible capabilities.

Development Roadmap

Phase 1

Infrastructure

- Project structure

- Configuration
- Logging
- FastAPI server
- Event Bus
- Authentication
- Device Registry

No AI.

Phase 2

Linux Agent

- Registration
- Heartbeat
- Capability discovery

Phase 3

Task System

- Task creation
- Queue
- Executor
- Result reporting

Phase 4

Dashboard

- Connected devices
- Logs
- Tasks
- System health

Phase 5

Model Manager

- Provider interface
- Ollama provider

- Model discovery
- Streaming

Phase 6

Planner

- Goal decomposition
- Tool selection
- Task generation

Phase 7

Memory

- User memory
- Knowledge
- Preferences

Phase 8

Additional Agents

- Android
- Windows
- macOS

Phase 9

Automation

Event-driven workflows.

Examples:

IF battery < 20%

THEN notify phone

Phase 10

Voice Interface

Whisper

↓

Planner

↓

Executor

↓

Piper

Minimum Viable Product

The first functional milestone is deliberately simple.

Scenario:

1. Jarvis Core starts.
2. Linux Agent connects.
3. Device registers successfully.
4. Server displays connected device.
5. User submits a command.
6. Server creates a task.
7. Agent executes the task.
8. Agent returns the result.
9. Server marks task complete.

No AI is required for Version 0.1.

Long-Term Vision

JarvisOS should evolve into an extensible personal intelligence platform capable of coordinating multiple AI models, multiple devices, and multiple execution environments while remaining local-first, secure, modular, and entirely open source.